

การเขียนโปรแกรม (Program Writing)

วิศวกรรมซอฟต์แวร์ (Software
Engineering)

รายชื่อสมาชิกกลุ่ม

1. อรปรียา อาสนทอง (6605001699)
2. อภิญญา วงษ์ศรี (6605005344)
3. กีมทีนา ลี (6705006069)
4. สุทธิดา ทรงรัมย์ (6605005716)
5. วิญญู สายแสง (6805007405)

บทนำ (Introduction)

- ✓ การเขียนโปรแกรมในงานวิศวกรรมซอฟต์แวร์ไม่ได้หมายถึงการเขียนคำสั่งให้โปรแกรมทำงานได้เพียงอย่างเดียว
- ✓ ต้องคำนึงถึงความอ่านง่าย ความสม่ำเสมอ การบำรุงรักษา การนำกลับมาใช้ใหม่ การตรวจสอบ และความสอดคล้องระหว่างการออกแบบกับโค้ดที่พัฒนา
- ✓ แนวทางเหล่านี้ช่วยให้ซอฟต์แวร์สามารถพัฒนาต่อได้ง่ายและลดปัญหาเมื่อนักพัฒนาหลายคนทำงานร่วมกัน

1.1 มาตรฐานต่างๆ สำหรับการเขียนโปรแกรม (Standards for Programming)

Coding Standards คือข้อตกลงหรือแนวทางกำหนดรูปแบบและวิธีการเขียนโค้ดให้เป็นมาตรฐานเดียวกันภายในทีมหรือองค์กร เพื่อให้โค้ดอ่านง่าย ตรวจสอบได้ง่าย และทำงานร่วมกันได้ดี

- **Naming Convention:** ตั้งชื่อ class, function, variable, constant และไฟล์ให้สื่อความหมายเดียวกัน
- **Formatting:** กำหนด indentation, เว้นวรรค, ขึ้นบรรทัด และการจัดวงเล็บให้สม่ำเสมอ
- **Code Organization:** กำหนดโครงสร้างไฟล์ โมดูล และฟังก์ชันให้เป็นระเบียบ
- **Comments & Documentation:** กำหนดเรื่องที่ต้องอธิบายด้วยคอมเมนต์และรูปแบบที่ถูกต้อง
- **Error Handling:** กำหนดแนวทางการตรวจสอบและจัดการข้อผิดพลาด
- **Quality & Safety:** กำหนดแนวปฏิบัติเพื่อลด defect และความเสี่ยงของซอฟต์แวร์

มาตรฐานและแนวทางปฏิบัติจริง

■ GNU Coding Standards

มุ่งให้โปรแกรมมีความสะอาด สอดคล้อง พกพาได้ และมีความน่าเชื่อถือ แม้จะเน้นภาษา C แต่หลักการหลายส่วนนำไปประยุกต์ใช้กับภาษาอื่นได้

🔗 IEEE Code Standards

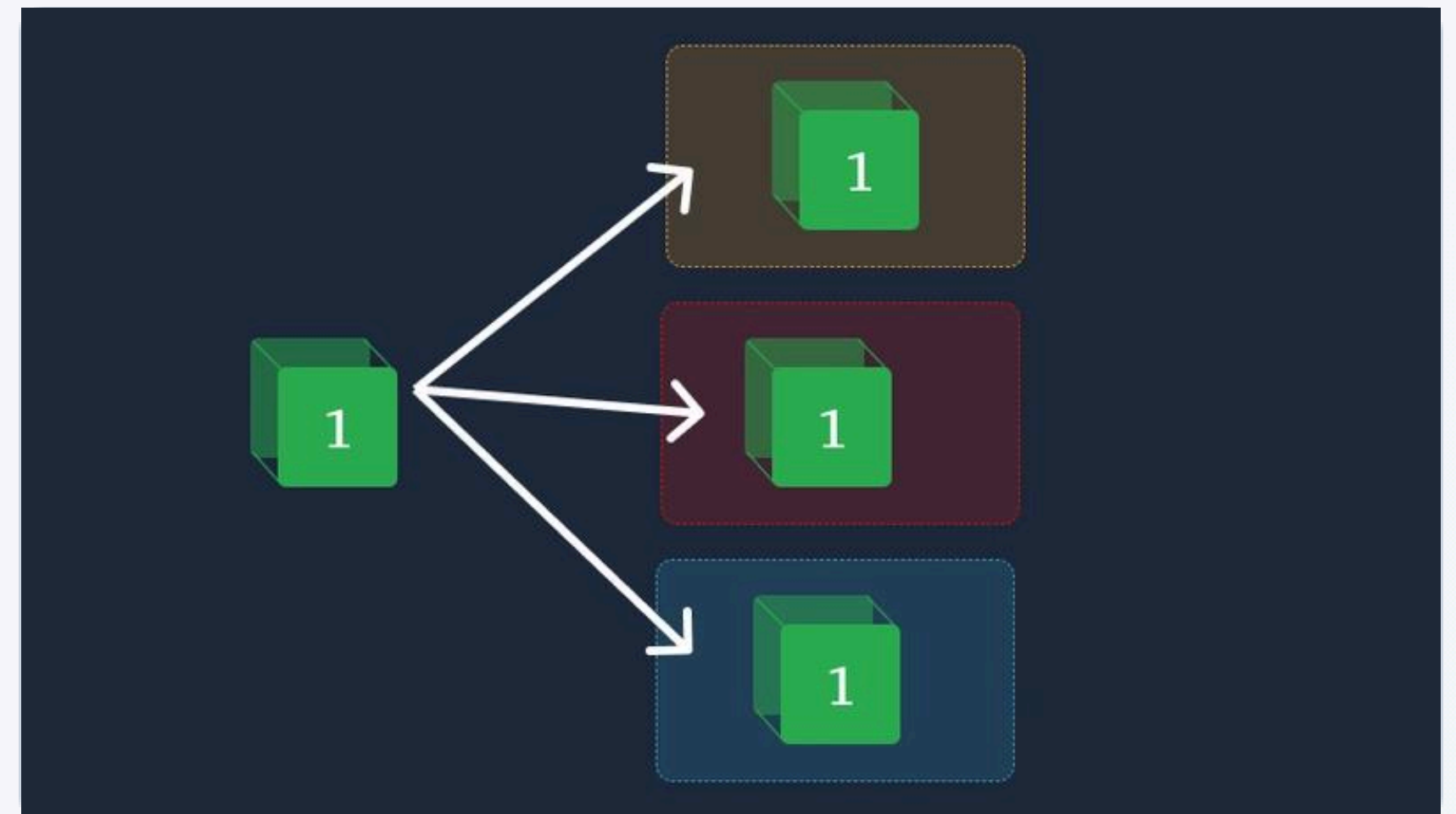
อธิบายว่า code standards ครอบคลุมข้อกำหนดด้าน naming, formatting, documentation และ safety เพื่อให้อ่านง่าย ดูแลรักษาได้ และทำงานร่วมกันได้ดี

สรุป: มาตรฐานการเขียนโปรแกรมไม่ได้มีไว้เพื่อบังคับรูปแบบ แต่เป็นเครื่องมือสร้างความสม่ำเสมอและลดต้นทุนในการอ่าน แก้ไข ทดสอบ และบำรุงรักษาโค้ด

1.2 แนวทางคำแนะนำสำหรับการนำกลับมาใช้ใหม่ (Guidelines for Reuse)

Software Reuse คือการนำ component หรือ library ที่มีอยู่แล้วมาใช้ในบริบทใหม่ แทนการพัฒนาจากศูนย์ ช่วยลดเวลาและต้นทุน แต่ต้องระวังเรื่อง dependency และความเข้ากันได้

1. **ค้นหาและประเมิน:** ตรวจสอบ library ที่มีอยู่ก่อนสร้างใหม่
2. **ออกแบบ Modular:** แยกส่วนประกอบให้มีหน้าที่ชัดเจน
3. **กำหนด Interface:** ระบุข้อมูลเข้า-ออกให้ชัดเจนโดยไม่ต้องรู้ภายใน



แนวทางสำคัญในการทำ Software Reuse (ต่อ)



4. ลด Coupling & Cohesion

ส่วนประกอบไม่ควรผูกติดกับ
บริบทเฉพาะมากเกินไป และ
มีหน้าที่เฉพาะเจาะจง



5. License & Compatibility

ตรวจสอบลิขสิทธิ์และความเข้า
กันได้ โดยเฉพาะเมื่อนำ Open-
Source มาใช้



6. พิจารณาค่าใช้จ่ายรวม

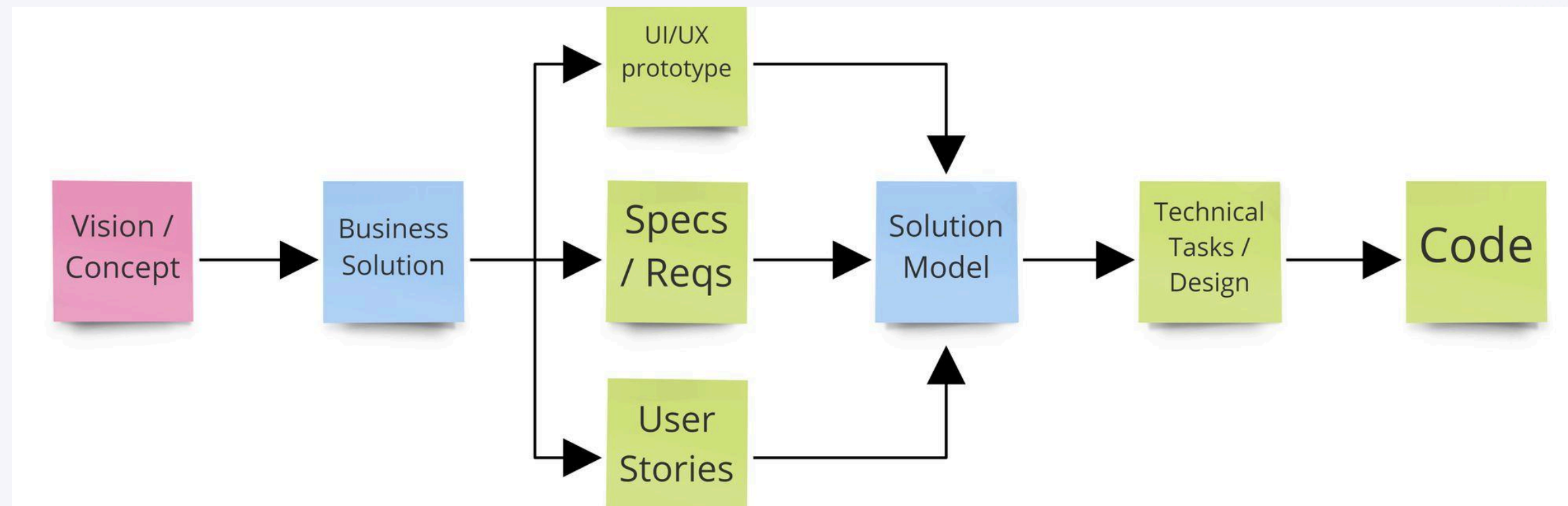
ประเมินต้นทุนในการทำ
ความเข้าใจ ดัดแปลง และดูแล
dependency ว่าคุ้มหรือไม่

งานวิจัยแบ่งการ reuse ได้หลายมิติ: Language-specific, Design-specific,
Domain-specific, Product-specific และ Architecture-specific

1.3 การใช้การออกแบบเป็นกรอบสำหรับการเขียนโปรแกรม

การออกแบบซอฟต์แวร์ทำหน้าที่เป็นกรอบก่อนเข้าสู่การเขียนโค้ด กำหนดส่วนประกอบ หน้าที่ และการติดต่อกัน เช่น ระบบชั้น **UI → Service → Data Access → Database**

- ช่วยให้โค้ดมีโครงสร้างตามที่ทีมออกแบบไว้
- ป้องกันการเขียนโค้ดแบบเฉพาะหน้า (Ad hoc coding)
- ทำให้การแก้ไขส่วนหนึ่งไม่กระทบส่วนอื่นโดยไม่จำเป็น



แนวทางปฏิบัติในการเชื่อมโยง Design กับ Code



Match Design

โครงสร้างของโค้ดควรสะท้อน
การออกแบบที่ได้รับอนุมัติอย่าง
เคร่งครัด



รักษา Abstraction

รายละเอียดภายใน module ไม่
ควรรั่วไหลไปยังส่วนอื่นโดยไม่
จำเป็น



ใช้ชื่อเดียวกัน

ชื่อ class, module, service
และ entity ในโค้ดควร
สอดคล้องกับ design

สรุป: Design เปรียบเสมือนพิมพ์เขียว ส่วน Code คือการก่อสร้าง ช่วยให้ระบบตรงกับความต้องการและ
สถาปัตยกรรมที่วางไว้

1.4 เอกสารภายในและเอกสารภายนอก (Documentation)

📄 Internal Documentation

เอกสารสำหรับทีมพัฒนาและผู้ดูแลระบบ เพื่อ
เข้าใจรายละเอียดภายในของซอฟต์แวร์

- Architecture & Design Document
- Comments อธิบายเหตุผลหรือ
implementation
- Coding conventions & API ภายใน

📄 External Documentation

เอกสารอธิบายการใช้งานแก่ผู้ใช้ นักพัฒนา
ภายนอก หรือผู้ติดตั้งระบบ

- User Manual & README
- API Documentation
- Installation & Configuration Guide

เปรียบเทียบ Internal และ External Documentation

ประเด็น	Internal Documentation	External Documentation
กลุ่มเป้าหมาย	นักพัฒนา ผู้ดูแล และทีมงานภายใน	ผู้ใช้ นักพัฒนาภายนอก หรือผู้ติดตั้งระบบ
เนื้อหา	รายละเอียด implementation และการพัฒนาภายใน	วิธีใช้งานและการเชื่อมต่อกับระบบ
ตัวอย่าง	Comments, Design Doc, Coding Guide	README, User Manual, API Guide
จุดประสงค์	ช่วยพัฒนา แก้ไข ทดสอบ และบำรุงรักษา	ช่วยให้ผู้ใช้หรือผู้พัฒนาภายนอกใช้งานระบบได้

สรุปภาพรวม (Summary)

- ★ การเขียนโปรแกรมที่ดีในวิศวกรรมซอฟต์แวร์ต้องพิจารณาองค์ประกอบรอบด้านอย่างเป็นระบบ
- ✓ **Standards:** สร้างความสม่ำเสมอและลดต้นทุนในการดูแลรักษาโค้ด
- ✓ **Reuse:** เลือกและออกแบบส่วนประกอบให้ใช้ซ้ำได้อย่างปลอดภัยและคุ้มค่า
- ✓ **Design:** ใช้สถาปัตยกรรมเป็นกรอบเพื่อป้องกันการเขียนโค้ดแบบเฉพาะหน้า
- ✓ **Documentation:** จัดทำเอกสารภายในและภายนอกให้สอดคล้องกับซอฟต์แวร์ปัจจุบัน